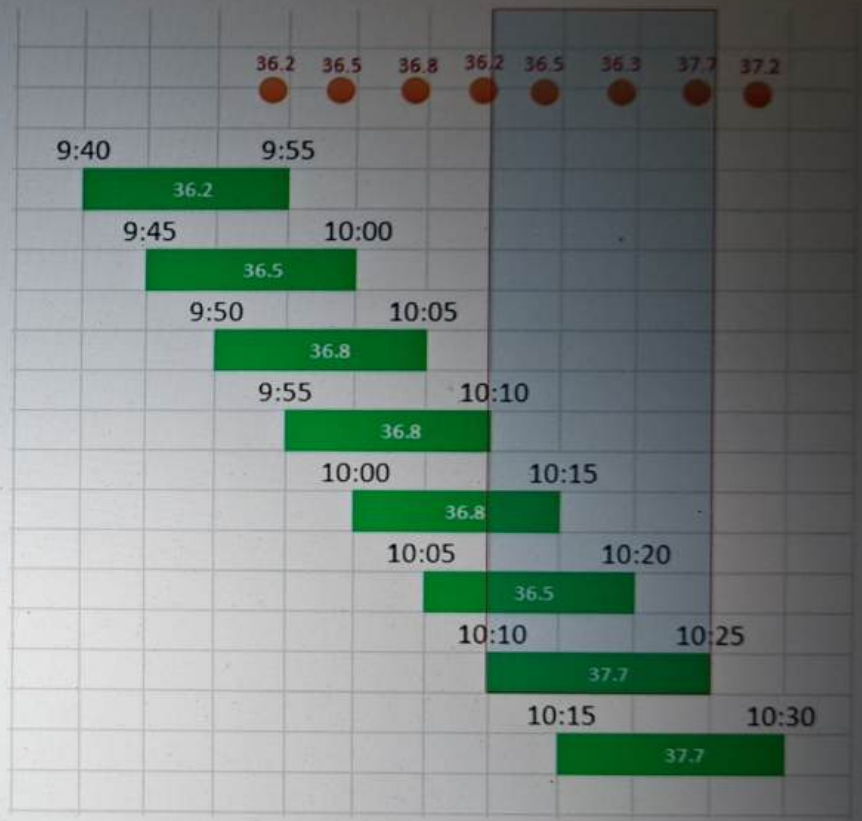
Time bound aggregates

1. Tumbling window
2. Sliding window

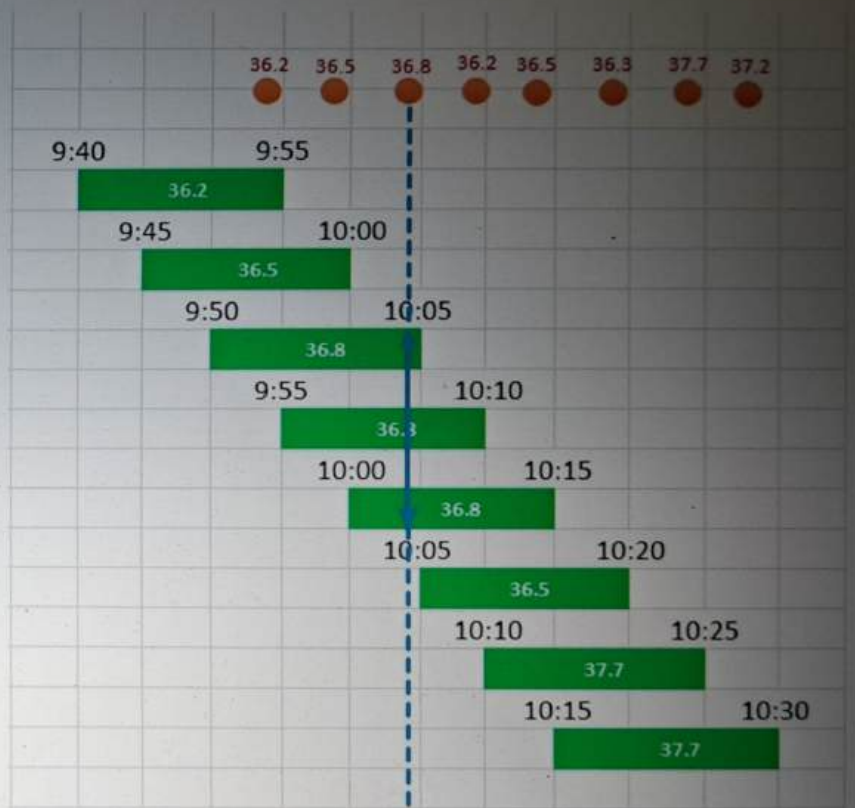
SET41:{"CreatedTime": "2019-02-05 09:54:00","Reading": 36.2}

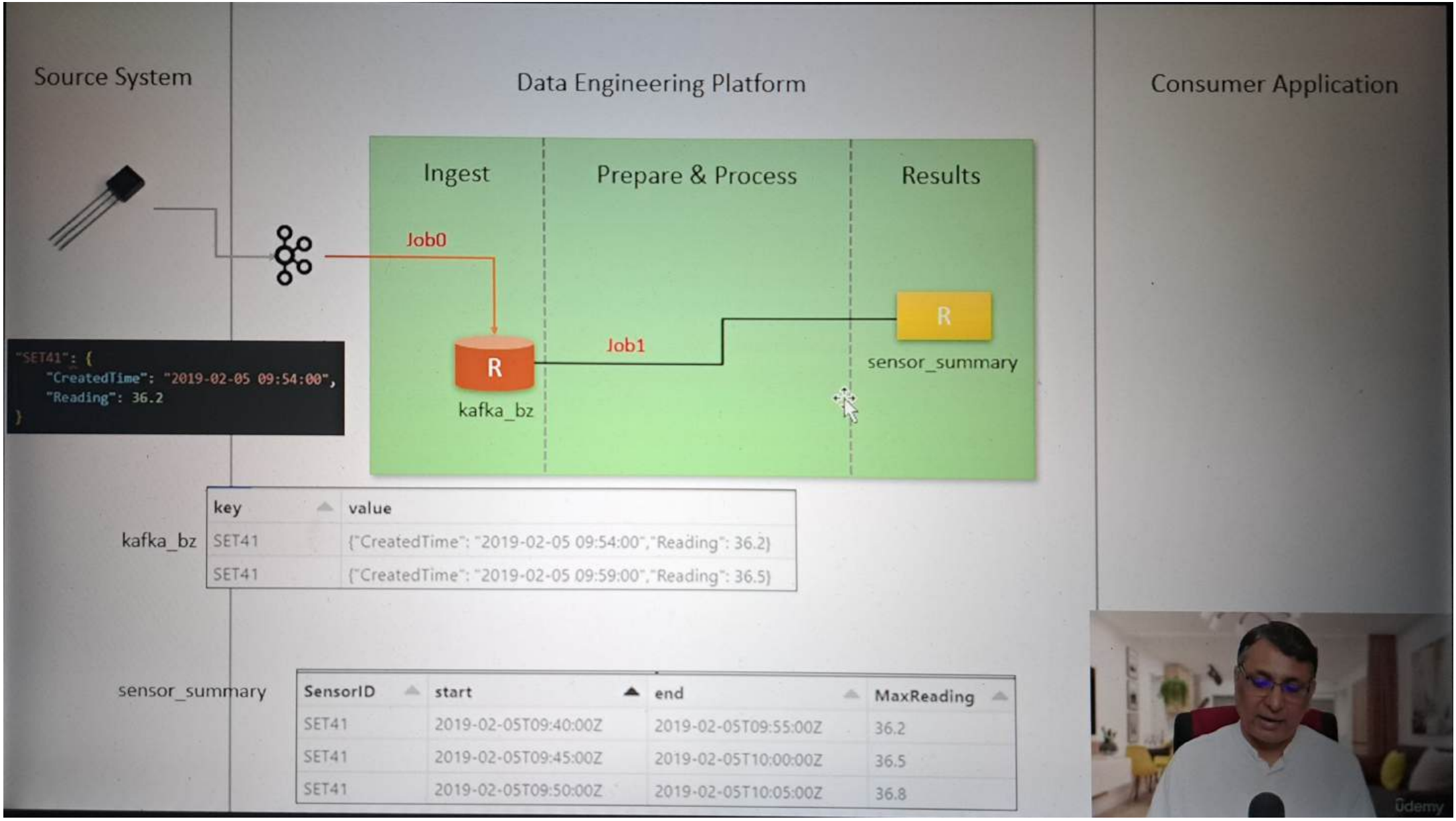
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3

SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3




Source System
Data Engineering Platform
Consumer Application
Ingest
Prepare & Process
Results
Job0
Job1
R
kafka_bz
R
sensor_summary
"SET41": {
"CreatedTime": "2019-02-05 09:54:00",
"Reading": 36.2
}
kafka_bz
key
value
SET41
{"CreatedTime": "2019-02-05 09:54:00","Reading": 36.2}
SET41
{"CreatedTime": "2019-02-05 09:59:00","Reading": 36.5}
sensor_summary
SensorID
start
end
MaxReading
SET41
2019-02-05T09:40:00Z
2019-02-05T09:55:00Z
36.2
SET41
2019-02-05T09:45:00Z
2019-02-05T10:00:00Z
36.5
SET41
2019-02-05T09:50:00Z
2019-02-05T10:05:00Z
36.8

key	value
SET41	{"CreatedTime": "2019-02-05 09:54:00","Reading": 36.2}
SET41	{"CreatedTime": "2019-02-05 09:59:00","Reading": 36.5}
SET41	{"CreatedTime": "2019-02-05 10:04:00","Reading": 36.8}
SET41	{"CreatedTime": "2019-02-05 10:09:00","Reading": 36.2}
SET41	{"CreatedTime": "2019-02-05 10:14:00","Reading": 36.5}
SET41	{"CreatedTime": "2019-02-05 10:19:00","Reading": 36.3}
SET41	{"CreatedTime": "2019-02-05 10:24:00","Reading": 37.7}
SET41	{"CreatedTime": "2019-02-05 10:29:00","Reading": 37.2}

SensorID	start	end	MaxReading
SET41	2019-02-05T09:40:00Z	2019-02-05T09:55:00Z	36.2
SET41	2019-02-05T09:45:00Z	2019-02-05T10:00:00Z	36.5
SET41	2019-02-05T09:50:00Z	2019-02-05T10:05:00Z	36.8
SET41	2019-02-05T09:55:00Z	2019-02-05T10:10:00Z	36.8
SET41	2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	36.8
SET41	2019-02-05T10:05:00Z	2019-02-05T10:20:00Z	36.5
SET41	2019-02-05T10:10:00Z	2019-02-05T10:25:00Z	37.7
SET41	2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	37.7
SET41	2019-02-05T10:20:00Z	2019-02-05T10:35:00Z	37.7
SET41	2019-02-05T10:25:00Z	2019-02-05T10:40:00Z	37.2

24-sliding-window Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Share Publish

Cmd 1

Python

```
class SlidingAggregate():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def getSchema(self):
        from pyspark.sql.types import StructType, StructField, StringType, DoubleType
        return StructType([
                    StructField("CreatedTime", StringType()),
                    StructField("Reading", DoubleType())
                ])

    def readBronze(self):
        return spark.readStream.table("kafka_bz")

    def getSensorData(self, kafka_df):
        from pyspark.sql.functions import from_json, expr
        return (kafka_df.select(kafka_df.key.cast("string").alias("SensorID"),
                                from_json(kafka_df.value.cast("string"), self.getSchema()).alias("value"))
                        .select("SensorID", "value.*")
                        .withColumn("CreatedTime", expr("to_timestamp(CreatedTime, 'yyyy-MM-dd HH:mm:ss')"
                )

    def getAggregate(self, sensor_df):
        from pyspark.sql.functions import window, max
        return (sensor_df.withWatermark("CreatedTime", "30 minutes")
```

```
    def readBronze(self):
        return spark.readStream.table("kafka_bz")

    def getSensorData(self, kafka_df):
        from pyspark.sql.functions import from_json, expr
        return (kafka_df.select(kafka_df.key.cast("string").alias("SensorID"),
                                from_json(kafka_df.value.cast("string"), self.getSchema()).alias("value"))
                        .select("SensorID", "value.*")
                        .withColumn("CreatedTime", expr("to_timestamp(CreatedTime, 'yyyy-MM-dd HH:mm:ss')"))
                )

    def getAggregate(self, sensor_df):
        from pyspark.sql.functions import window, max
        return (sensor_df.withWatermark("CreatedTime", "30 minutes")
                        .groupBy(sensor_df.SensorID,
                                 window(sensor_df.CreatedTime, "15 minutes"))
                        .agg(max("Reading").alias("MaxReading"))
                        .select("SensorID", "window.start", "window.end", "MaxReading")
                )

    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                        .queryName("sensor-query")
                        .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/sensor_summary")
                        .outputMode("complete")
                        .toTable("sensor_summary")
                )
```

```
        return (kafka_df.select(kafka_df.key.cast("string").alias("SensorID"),
                                from_json(kafka_df.value.cast("string"), self.getSchema()).alias("value"))
                        .select("SensorID", "value.*")
                        .withColumn("CreatedTime", expr("to_timestamp(CreatedTime, 'yyyy-MM-dd HH:mm:ss')"))
                )

    def getAggregate(self, sensor_df):
        from pyspark.sql.functions import window, max
        return (sensor_df.withWatermark("CreatedTime", "30 minutes")
                        .groupBy(sensor_df.SensorID,
                                 window(sensor_df.CreatedTime, "15 minutes", "5 minutes"))
                        .agg(max("Reading").alias("MaxReading"))
                        .select("SensorID", "window.start", "window.end", "MaxReading")
                )

    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                        .queryName("sensor-query")
                        .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/sensor_summary")
                        .outputMode("complete")
                        .toTable("sensor_summary")
                )
        print("Done")

    def process(self):
        kafka_df = self.readBronze()
        sensor_df = self.getSensorData(kafka_df)
```

```
                .groupBy(sensor_df.SensorID,
                         window(sensor_df.CreatedTime, "15 minutes", "5 minutes"))
                .agg(max("Reading").alias("MaxReading"))
                .select("SensorID", "window.start", "window.end", "MaxReading")
            )

    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                    .queryName("sensor-query")
                    .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/sensor_summary")
                    .outputMode("complete")
                    .toTable("sensor_summary")
                )
        print("Done")

    def process(self):
        kafka_df = self.readBronze()
        sensor_df = self.getSensorData(kafka_df)
        results_df = self.getAggregate(sensor_df)
        sQuery = self.saveResults(results_df)
        return sQuery
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

24-sliding-window-test-suite Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Share Publish

```
%run ./24-sliding-window
```

Cmd 2

Python

```
class SensorSummaryTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists kafka_bz")
        spark.sql("drop table if exists sensor_summary")
        dbutils.fs.rm("/user/hive/warehouse/kafka_bz", True)
        dbutils.fs.rm("/user/hive/warehouse/sensor_summary", True)
        spark.sql(f"CREATE TABLE kafka_bz(key STRING, value STRING)")

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/sensor_summary", True)
        print("Done")

    def waitForMicroBatch(self, sleep=60):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")
```

```
    def waitForMicroBatch(self, sleep=60):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")

    def assertSensorSummary(self):
        print(f"\tStarting Sensor Summary validation...", end='')
        actual_result = spark.table("sensor_summary").collect()

        expected_result = (spark.read.format("csv")
                               .option("header", "true")
                               .load(f"{self.base_data_dir}/datasets/results/sliding_window_result.csv")
                               .collect()
                           )

        assert expected_result == actual_result, f"Test failed! actual resutl is {actual_result}"
        print("Done")
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

24-sliding-window-test-suite Python

File Edit View Run Help Last edit was 8 minutes ago Provide feedback

Run all Connect Share Publish

```
    def runTests(self):
        self.cleanTests()

        stream = SlidingAggregate()
        sQuery = stream.process()

        print("\nTesting all events...")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

24-sliding-window-test-suite Python

File Edit View Run Help Last edit was 9 minutes ago Provide feedback

Run all Connect Share Publish

```
print("\nTesting all events...")
spark.sql("""INSERT INTO kafka_bz VALUES
        ('SET41', '{"CreatedTime": "2019-02-05 09:54:00","Reading": 36.2}'),
        ('SET41', '{"CreatedTime": "2019-02-05 09:59:00","Reading": 36.5}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:04:00","Reading": 36.8}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:09:00","Reading": 36.2}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:14:00","Reading": 36.5}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:19:00","Reading": 36.3}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:24:00","Reading": 37.7}'),
        ('SET41', '{"CreatedTime": "2019-02-05 10:29:00","Reading": 37.2}')
    """)
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

24-sliding-window-test-suite Python

File Edit View Run Help Last edit was 11 minutes ago Provide feedback

Run all Connecting Share Publish

```
                ('SET41', '{"CreatedTime": "2019-02-05 10:24:00","Reading": 37.7}'),
                ('SET41', '{"CreatedTime": "2019-02-05 10:29:00","Reading": 37.2}')
            """)

        print("Validation passed.\n")
        sQuery.stop()
```

Cmd 3

Python

```
ts = SensorSummaryTestSuite()
ts.runTests()
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text